Aide mémoire Python

Les bases

Mots-clefs

and as assert break class continue def del elif else except finally for from global if import in is lambda not or pass raise return try while with yield

Aide – Entrées/sorties

• **help()** aide interactive • **help(o)** aide spécifique sur o • **dir()** liste des noms connus • **dir(o)** liste des noms associés à o • **input(**prompt**)**→str : saisie • **print(**o1,o2,**sep=**ch1,**end=**ch2**)**→None : affichage

Fonctions prédéfinies

abs all any ascii bin bool bytearray bytes chr classmethod compile complex copyright credits delattr dict dir divmod enumerate eval exec exit filter float format frozenset getattr globals hasattr hash help hex id input int isinstance issubclass iter len license list locals map max memoryview min next object oct open ord pow print property quit range repr reversed round set setattr slice sorted staticmethod str sum super tuple type vars zip

Exceptions prédéfinies

ArithmeticError AssertionError AttributeError BufferError EOFError EnvironmentError FloatingPointError IOError ImportError IndentationError IndexError KeyError LookupError MemoryError NameError NotImplementedError OSError OverflowError ReferenceError RuntimeError SyntaxError SystemError TabError TypeError UnboundLocalError UnicodeDecodeError UnicodeEncodeError UnicodeError UnicodeTranslateError ValueError WindowsError ZeroDivisionError

Types prédéfinis

Types numériques

bool sous-classe de int True(==1) False(==0)
Expressions logiques (sur objets bool) • **not** b→bool • b1&b2 → bool • b1|b2→bool • b1^b2→bool
Expressions logiques (sur objets quelconques)
bool(x**)**==False si x (numérique) est égal à 0 ou si x (conteneur) est vide
bool(x**)**==True si x (numérique) est différent de 0 ou si x (conteneur) est non vide
• x **and** y → x si bool(x) False, →y sinon • x **or** y → x si bool(x) True, →y sinon
• **not** x → True si bool(x) False, → False sinon
o1 **if** x **else** o2 → o1 si bool(x) True, → o2 sinon

int entiers • 123 (base 10) • 0b111 (base 2) • 0o17 (base 8) • 0x1FA (base 16) (pas de limite)

float décimaux • 3.14 • 2.1e-23 (≈16 chiffres significatifs, de 10^{-324} à 10^{308})

complex complexes • 3.1+5.1j

Opérations sur les numériques

• x **or** y : disjonction logique • x **and** y : conjonction logique • **not** x : négation logique • x**<**(2)y x**<=**(2)y x**>**(2)y x**>=**(2)y x**!=**y x**==**y : comparaisons • x**|**(1)y : disjonction bit à bit • x**^**(1)y : disjonction exclusive bit à bit • x**&**(1)y :conjonction bit à bit • x**<<**(1)y x**>>**(1)y : décalages (gauche et droite) bit à bit • x**+**y x**-**y : addition, soustraction • x*****y x/y x**//**(2)y x**%**(2)y : multiplication, divisions(float et int), reste • **+**x **-**x **~**(1)x : plus et moins unaires, négation bit à bit • x******y : élévation à la puisance • **abs(**x**)** : valeur absolue (ou module)
• **divmod**(2)**(**x,y**)** : couple (quotient, reste) • **round**(2)**(**x,déc**)** : arrondi
(1) bool et int uniquement (2) bool, int et float uniquement

Conteneurs

Donnée itérable composée de zéro ou plusieurs objets
• **len(**cont**)**→int • x **in** cont→bool • x not **in** cont→bool • **for** x **in** cont :

tuple séquence non modifiable d'objets quelconques indicée à partir de 0 et découpable

Création • **()** • **(**o,**)** • **(**o1,o2,…**)** • **tuple(**iter**)** • **tuple(**f(o) **for** o **in** iter**)**
Extraction • t**[**i**]**→object • t**[**début:fin:pas**]**→tuple
Recherche • t.**index(**objet,début,fin**)**→int • t.**count(**objet,début,fin**)**→int
Concaténation • t1**+**t2→tuple • t*****n→tuple
Booléennes • t1==t2→bool • t1**!=**t2→bool • t1**<=**t2→bool • t1**<**t2→bool • t1**>=**t2→bool • t1**>**t2→ bool

list séquence modifiable d'objets quelconques indicée à partir de 0 et découpable

Création • **[]** • **list()** • **[**o1, o2, …**]** • **list(**iter**)** • **list(**f(o) **for** o **in** iter**)** • **[**f(o) **for** o **in** iter**]** • l**[:]**
Extraction • l**[**i**]**→object • l**[**début:fin:pas**]**→list
Recherche • l.**index(**objet,début,fin**)**→int • l.**count(**objet,début,fin**)**→int
Concaténation • l1**+**l2→list • l*****n→list
Booléennes • l1==l2→bool • l1**!=**l2→bool • l1**<=**l2→bool • l1**<**l2→bool • l1**>=**l2→bool • l1**>**l2→bool
Modification • l1**+=**l2 • l1***=**l2 • l**[**i**] =** object • l**[**début : fin : pas**] =** iter_obj • l.**append(**object**)** • l.**extend(**iter_obj**)** • l.**insert(**pos, object**)** • l.**pop(**i**)** • l.**remove(**value**)** • l.**reverse()** • l.**sort(**key=None, reverse=False**)**

str séquence non modifiable de caractères non modifiables indicée à partir de 0, découpable

Création • **""** • **"hello"** • **'hello'** • **"""hello"""** • **'a\nb'** • **"\ta\\b"** **"\u00e9"** • **str(**o**)** • ch**[:]**
Extraction • ch**[**i**]**→str • ch**[**début:fin:pas**]**→str
Partition • ch.**partition(**sep**)**→(chdéb,sep,chfin)
• ch.**rpartition(**sep**)**→(chdéb,sep,chfin) • ch.**split(**sep, maxsplit**)**→list_ch
• ch.**rsplit(**sep,maxsplit**)**→list_ch • ch.**splitlines(**keepend=False**)**→list_ch
Recherche/ remplacement • ch.**count(**ssch,début,fin**)**→int • ch.**find(**ssch,début,fin**)**→int
• ch.**rfind(**ssch,début,fin**)**→int • ch.**index(**ssch,début,fin**)**→int
• ch.**rindex(**ssch,début,fin**)**→int • ch.**replace(**ssch1,ssch2,nb**)**→str
Concaténation • ch1**+**ch2→str • ch*****n→str • ch.**join(**iter_ch**)**→str
Formattage • ch.**lower()**→str • ch.**upper()**→str • ch.**swapcase()**→str • ch.**title()**→str
• ch.**capitalize()**→str • ch.**center(**larg,car**)**→str • ch.**ljust(**larg,car**)**→str
• ch.**rjust(**larg,car**)**→str • ch.**zfill(**larg**)**→str • ch.**strip(**cars**)**→str
• ch.**lstrip(**cars**)**→str • ch.**rstrip(**cars**)**→str • ch.**expandtabs(**larg**)**→str
• ch.**format(***args,**motcles**)**→str
Booléennes • ch1==ch2→bool • ch1**!=**ch2→bool • ch1**<=**ch2→bool • ch1**<**ch2→bool
• ch1**>=**ch2→bool • ch1**>**ch2→bool • ch.**startswith(**ssch,début,fin**)**→bool
• ch.**endswith(**ssch,début,fin**)** →bool • ch.**isalnum()** →bool • ch.**isalpha()** →bool
• ch.**isnumeric()** →bool • ch.**isdecimal()** →bool • ch.**isdigit()** →bool
• ch.**isspace()** →bool • ch.**isprintable()** →bool • ch.**isidentifier()** →bool
• ch.**islower()** → bool • ch.**isupper()** → bool • ch.**istitle()**→bool
Diverses • chn.**encode(**encoding,errors**)**→bytes • **str.maketrans(**x,y,z**)**→dict
• chn.**translate(**table**)**→str

set ensemble sans doublons, non indicé et modifiable d'objets non modifiables

Création ● **set()** ● **set(**o1,o2,…**)** ● **set(**iter**)** ● set(f(o) **for** o **in** iter)
● **{**f(o) **for** o **in** iter**}** ● e.**copy()**
Opérations ensemblistes ● e1**<**e2→bool ● e1**<=**e2→bool ● e1.**issubset(**e2**)**→bool
● e1**>**e2→bool ● e1**>=**e2→bool ● e1.**issuperset(**e2**)**→bool ● e1.**isdisjoint(**e2**)**→bool
● e1**&**e2→set ● e1.**intersection(**e2**)**→set ● e1**|**e2→set ● e1.**union(**e2**)**→set ● e1-e2→set
● e1.**difference(**e2**)**→set ● e1^e2→set ● e1.**symmetric_difference(**e2**)**→set
Comparaisons ● e1==e2 ●e1!=e2
Modification ● e1**&=**e2 ● e1.**intersection_update(**e2**)**● e1**|=**e2 ● e1.**update(**e2**)**
● e1**-=**e2 ● e1.**difference_update(**e2**)** ● e1**^=**e2
● e1.**symmetric_difference_update(**e2**)**
● e.**clear()** ● e.**add(**o**)** ● e.**discard(**o**)** ● e.**remove(**o**)** ● e.**pop()**→object

dict ensemble non indicé, modifiable, de couples <*clé, valeur*> sans doublons de clé

Création ● **{}** ● **dict()** ● **dict(**iter_de_couples**)** ● **dict(**map_de_couples**)**
● **dict(**k1=v1,k2=v2,…**)** ● d.**copy()** ● d.**fromkeys(**iter,v**)**
Extraction ● d**[**k**]**→object ● d.**get(**k,o**)**→object
Vues ● d.**items()**→dict_items ● d.**keys()**→dict_keys ● d.**values()**→dict_values
Comparaisons ● d1==d2 ● d1!=d2
Modification ● d**[**k**]=**o ● d.**pop(**k,o**)**→object ● d.**popitem()**→(k,v)
● d.**setdefault(**k,o**)**● d.**clear()** ● **del** d**[**k**]** ● d1.**update(**d2,k1=v1,k2=v2,…**)**
● d1.**update(**iter_de_couples,k1=v1,k2=v2,…**)**

Itérateurs

Objets parcourables une seule fois. (**for** x **in** iterateur **:**) **générant une nouvelle valeur à chaque tour de boucle . Les conteneurs sont leurs propres itérateurs.**

range(déb,fin,p**)** : génère les nombre de déb (=0) à fin (exclus) par pas de p (=1)
reversed(iter**)** genère les iter[i] à rebours
zip(iter0,iter1,…**)** génère les tuple (iter0[i],iter1[i],…)
enumerate(iter**)** génère les couples (i,iter[i])
filter(f_ou_None,iter**)** génère les iter[i] pour lesquels f(iter[i]) (ou iter[i] est vrai)
map(f,iter0,iter1,…**)** génère les f(iter0[i],iter1[i],…)

Fonctions sur itérables

si les éléments de iter ont une valeur booléenne
● **all**(iter)→bool ● **any**(iter)→bool
si les éléments de iter sont comparables
● **max**(iter,**key=**f)→object ● **min**(iter,**key=**f)→object
● **sorted**(iter,**key=**None,**reverse=**False)→list
si les éléments de iter sont des nombres
sum(iter,val0])→object

Fichiers texte

```
f = open(chaîne-nom, chaîne-type-ouverture,encodage)
f.close(...) f.detach(...) f.fileno(...) f.flush(...) f.isatty(...) f.read(...)
f.readable(...) f.readline(...) f.seek(...) f.seekable(...) f.tell(...)
f.truncate(...) f.writable(...) f.write(...)
f.readlines(...) f.writelines(...)
```

Programmation

Instructions simples

● expressions (calcul, appels fonction) ● x=expr : affectation● x**+=**expr x**-=**expr x***=**expr x**/=**expr x**//=**expr x**%=**expr x****=**expr x**>>=**expr x***<<=**expr x**&=**expr x***^=**expr x**|=**expr : affectation augmentée● **assert** expr1,expr2,... : assertion ● **pass** : instruction vide ● **del** x : suppression ● **return** expr : sortie fonction (avec retour valeur) ● **yield** expr : production d'une valeur ● **raise** exception : lancement d'une exception ● **break** : sortie de boucle ● **continue** : prochaine itération dans une boucle ● **import** module : importation module

Instructions composées

if (conditionnelle)

```
if cond :
    bloc-if
elif cond-i : # 0/n
    bloc-i
else : # 0/1
    bloc-else
```

while (boucle tant que)

```
while cond :
    bloc-while
else :  # 0/1
    bloc-else # fin de boucle
```

for (boucle pour)

```
for x1,...,xi,... in iter :
    bloc-for
else : # 0/1
    bloc-else # fin de boucle
```

try (protection par exception)

```
try :
    bloc-protégé
except typeException-i as e-i : #0/n
    bloc-i
else : # 0/1
    bloc-else
finally : # 0/1
    bloc-finally
```

Définitions

def (définition de fonction/méthode)

```
def f(arg) :
    bloc_def
    return expr
```

def (définition de générateur)

```
def f(arg) :
    bloc_def
    yield expr
```

class (définition de classe)

```
class C(arg) :
    bloc_def
#définir ici les méthodes du genre :
#__init__  __del__  __repr__  __str__
#__lt__  __le__  __eq__  __ne__
#__gt__  __ge__  __bool__
#__add__  __radd__  __iadd__
```

Principaux modules

● **fraction decimal** : autres types numériques ● **array collections** : autres type de conteneurs ● **sys glob shutil os** : interaction avec le système d'exploitation ● **getopt optparse** : arguments de laligne de commande ● **re** : gestion d'expressions régulières ● **math random** : mathématiques, générateurs aléatoires ● **urllib.request smtplib email** : internet, email ● **datetime** : dates et heures ● **zlib gzip bz2 zipfile tarfile.**: compression de données **profile pstats timeit** : mesure des performances ● **doctest unittest** : tests de programmes ● **csv xml.dom xml.sax** : gestionde format de fichiers ● **gettext locale codecs** : internationalisation ● **reprlib pprint textwrap** : outils de formattage ● **struct** : gestion de données en format binaire ● **threading** : gestion de threads